

LAPORAN PRAKTIKUM
PRAKTIKUM PEMROGRAMAN WEBSITE 1
PERTEMUAN 7
“STYLESHEET 2”



Disusun Oleh:

Nama	: Misbahul Huda
NIM	: 25/568209/SV/27391
Kelas	: PL2B2
Dosen Pengampu	: Achmad Choirudin Emcha, S.Kom., M.Eng.

PROGRAM STUDI D IV TEKNOLOGI REKAYASA PERANGKAT
LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

PERTEMUAN 7

STYLESHEET 2

1. Tujuan

- A. Memahami cara mendefinisikan dan menggunakan CSS Variables sebagai sistem palet warna terpusat yang dapat digunakan secara konsisten di seluruh file.
- B. Mengimplementasikan properti transition pada elemen tombol untuk menciptakan efek hover yang responsif, serta membuat animasi menggunakan @keyframes dan animation untuk komponen loading spinner dan efek fade-in saat halaman dibuka.
- C. Menerapkan properti transformasi CSS tiga dimensi seperti transform: rotateY() untuk membangun komponen flip card yang interaktif dengan memanfaatkan perspective, transform-style, dan backface-visibility.

2. Dasar Teori

2.1. Pengertian Umum

Stylesheet adalah berkas atau sekumpulan aturan (rules) yang digunakan untuk mengontrol tampilan (presentasi) elemen-elemen pada dokumen web. Teknologi stylesheet yang paling umum digunakan adalah Cascading Style Sheets (CSS). W3C menjelaskan CSS sebagai bahasa style sheet yang memungkinkan penulis melampirkan gaya seperti font, warna, dan spasi pada dokumen terstruktur seperti HTML [1]. CSS dirancang untuk memisahkan konten dari presentasi, sehingga struktur halaman (HTML) dan tampilannya tidak bercampur [1][2].

CSS menyediakan mekanisme sederhana untuk mengatur tampilan halaman web, misalnya pengaturan warna, ukuran huruf,

margin, padding, dan tata letak visual [3]. Dengan pemisahan ini, pengembang dapat mengelola tampilan secara lebih efisien, konsisten, serta mudah diubah tanpa menyentuh konten inti halaman.

2.2. Implementasi

Implementasi dasar stylesheet pada pembuatan website dilakukan dengan memisahkan aturan tampilan dari konten menggunakan berkas CSS. Pada praktik umum, developer membuat sebuah file eksternal seperti `style.css` yang berisi aturan visual, lalu menghubungkannya ke halaman HTML melalui elemen `<link>` di dalam bagian `<head>` halaman tersebut [5]. Cara ini adalah yang paling disarankan karena memungkinkan tampilan dikenali sebagai modul terpisah, dapat digunakan kembali pada banyak halaman, dan memanfaatkan cache browser sehingga mempercepat proses pemuatan halaman bagi pengguna. Di dalam berkas stylesheet tersebut, pengembang mendefinisikan selektor—misalnya kelas, ID, atau nama elemen—yang diikuti oleh properti dan nilai, seperti warna, ukuran huruf, atau jarak antar elemen, untuk mengatur bagaimana konten ditampilkan [1]. Selain itu, implementasi CSS juga mengikuti prinsip *cascading* dan *spesifisitas*, di mana aturan yang lebih spesifik atau ditulis lebih akhir memiliki prioritas lebih tinggi ketika beberapa aturan saling bertentangan [6]. Dalam proses ini, seluruh elemen yang dirender oleh browser diperlakukan sebagai bagian dari *box model* yang mencakup *content*, *padding*, *border*, dan *margin*, sehingga pengembang dapat mengatur tata letak dan proporsi ruang antarelemen secara sistematis [7]. Dengan memanfaatkan konsep-konsep ini, stylesheet memungkinkan tampilan website tetap konsisten, terstruktur, dan mudah diubah tanpa harus memodifikasi konten HTML secara langsung.

Berikut adalah contoh Implementasinya dalam bentuk codingannya:

a) File html (index.html)

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Contoh Stylesheet</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

  <h1 class="judul">Selamat Datang di Website
Saya</h1>
  <p class="deskripsi">
    Ini adalah contoh bagaimana stylesheet
digunakan untuk mengatur tampilan halaman web.
  </p>

  <div class="kartu">
    <h2>Fitur Utama</h2>
    <p>Stylesheet memungkinkan pengembang
mengatur warna, jarak, ukuran font, dan tata
letak.</p>
  </div>

</body>
</html>
```

b) File css (style.css)

```
body {
  font-family: Arial, sans-serif;
  background-color: #f4f4f4;
  margin: 0;
  padding: 20px;
}

.judul {
  color: #2c3e50;
```

```

        text-align: center;
    }

    .deskripsi {
        font-size: 18px;
        text-align: center;
        color: #333;
    }

    .kartu {
        background-color: white;
        padding: 20px;
        border-radius: 8px;
        width: 60%;
        margin: 20px auto;
        box-shadow: 0 4px 8px rgba(0,0,0,0.1);
    }

```

Pada contoh di atas, implementasi stylesheet dilakukan dengan memisahkan aturan tampilan ke dalam file terpisah bernama style.css. HTML hanya berfungsi sebagai struktur konten, sementara CSS bertugas mengontrol aspek visualnya, sesuai prinsip pemisahan konten dan presentasi seperti yang direkomendasikan W3C [1][2]. File HTML memanggil stylesheet melalui elemen `<link rel="stylesheet" href="style.css">` di dalam bagian `<head>`, yang merupakan metode external stylesheet—cara terbaik untuk pemeliharaan dan pengelolaan tampilan halaman web dalam jangka panjang [5].

Dalam file CSS, aturan-aturan gaya ditulis dengan format: selektor → properti → nilai. Misalnya, selektor `.judul` mengatur teks judul agar berwarna biru gelap dan berada di tengah halaman, sedangkan `.kartu` mengatur tampilan sebuah kotak (card) dengan latar belakang putih, padding 20px, radius sudut 8px, serta bayangan untuk memberikan efek elevasi. Semua elemen HTML

yang memiliki kelas atau selector yang sesuai akan mengikuti aturan yang ditetapkan.

Selain itu, browser menerapkan konsep cascading dan spesifisitas, sehingga jika terdapat aturan yang bertentangan, browser akan menentukan aturan yang paling tepat berdasarkan hierarki prioritas [6]. Pada elemen seperti `div.kartu`, seluruh properti tampilan juga mengikuti CSS Box Model yang meliputi konten, padding, border, dan margin, di mana semua bagian ini bersama-sama menentukan ukuran total elemen pada halaman [7]. Dengan implementasi ini, developer dapat menjaga konsistensi desain, mempermudah pemeliharaan, dan mempercepat pengembangan tanpa harus menulis ulang aturan untuk setiap halaman, sesuai praktik terbaik dalam pengembangan web modern.

3. Hasil & Pembahasan

3.1. Tugas 1 CSS Position -> Layout Kartu Profil

Buat halaman HTML dengan elemen-elemen berikut menggunakan CSS Position: Buat sebuah kartu profil dengan position: relative sebagai parent container (lebar 280px, tinggi 350px) Di dalam kartu, tambahkan badge status ("Online ●") dengan position: absolute di sudut kanan atas Tambahkan sticky navbar di bagian atas halaman yang tetap terlihat saat scroll Tambahkan tombol "Back to Top" dengan position: fixed di pojok kanan bawah ★ Bonus: Tambahkan z-index agar elemen yang seharusnya di depan tidak tertutup elemen lain.

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
```

```
<title>Tugas 1 - CSS Position</title>
<style>
  * {
    box-sizing: border-box;
    scroll-behavior: smooth;
  }

  body {
    margin: 0;
    font-family: Arial, sans-serif;
    background: #f5f7fb;
    color: #222;
  }

  .navbar {
    position: sticky;
    top: 0;
    z-index: 1000;
    background: #1f2937;
    color: white;
    padding: 16px 24px;
    display: flex;
    justify-content: space-between;
    align-items: center;
    box-shadow: 0 2px 10px rgba(0, 0, 0, 0.15);
  }

  .navbar .logo {
    font-size: 20px;
    font-weight: bold;
  }

  .navbar a {
    color: white;
    text-decoration: none;
    margin-left: 16px;
    font-size: 14px;
  }

  .container {
    max-width: 1100px;
```

```
margin: 0 auto;
padding: 40px 20px 100px;
}

.title {
  text-align: center;
  margin-bottom: 30px;
}

.profile-card {
  position: relative;
  width: 280px;
  height: 350px;
  margin: 0 auto;
  background: white;
  border-radius: 18px;
  overflow: hidden;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.12);
  z-index: 1;
}

.profile-card img {
  width: 100%;
  height: 180px;
  object-fit: cover;
  display: block;
}

.profile-content {
  padding: 18px;
  text-align: center;
}

.profile-content h2 {
  margin: 10px 0 8px;
  font-size: 22px;
}

.profile-content p {
  margin: 0;
  font-size: 14px;
}
```



```
    color: #555;
    line-height: 1.5;
}

.badge {
    position: absolute;
    top: 12px;
    right: 12px;
    background: rgba(34, 197, 94, 0.95);
    color: white;
    padding: 8px 12px;
    border-radius: 999px;
    font-size: 13px;
    font-weight: bold;
    z-index: 2;
}

.spacer {
    height: 900px;
    padding-top: 50px;
    text-align: center;
    color: #666;
}

.back-to-top {
    position: fixed;
    right: 20px;
    bottom: 20px;
    background: #2563eb;
    color: white;
    text-decoration: none;
    padding: 12px 16px;
    border-radius: 999px;
    box-shadow: 0 8px 20px rgba(37, 99, 235, 0.35);
    z-index: 999;
    font-size: 14px;
}

.back-to-top:hover {
    background: #1d4ed8;
}
```

```

    </style>
</head>
<body id="top">

    <nav class="navbar">
        <div class="logo">MyProfile</div>
        <div class="menu">
            <a href="#top">Home</a>
            <a href="#profile">Profile</a>
            <a href="#contact">Contact</a>
        </div>
    </nav>

    <div class="container">
        <h1 class="title">CSS Position – Layout Kartu
        Profil</h1>

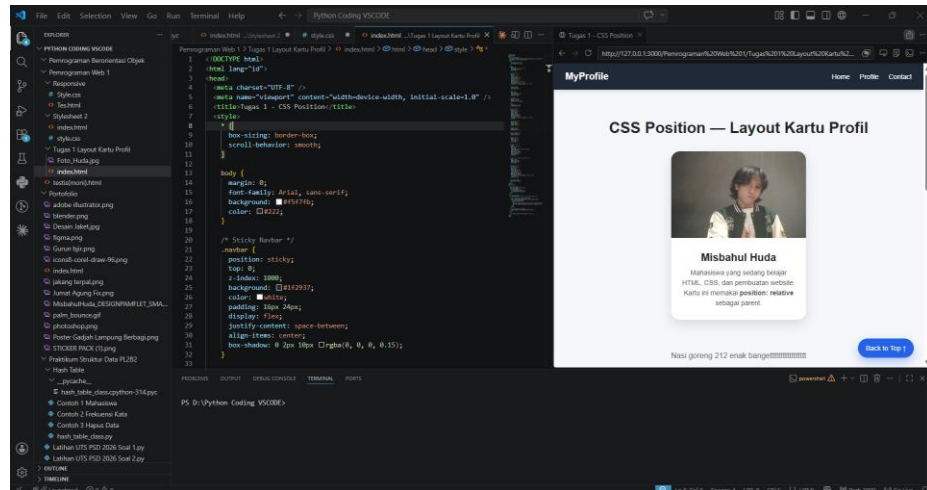
        <section id="profile" class="profile-card">
            
            <div class="profile-content">
                <h2>Misbahul Huda</h2>
                <p>
                    Mahasiswa yang sedang belajar HTML, CSS, dan
                    pembuatan website.
                    Kartu ini memakai <b>position: relative</b>
                    sebagai parent.
                </p>
            </div>
        </section>

        <div class="spacer" id="contact">
            <p>Nasi goreng 212 enak bangettttttttttttttttt</p>
        </div>

        <a class="back-to-top" href="#top">Back to Top ↑</a>
    </body>
</html>

```

Outputnya sebagai berikut:



Gambar 2.1 Tampilan output layout kartu profil

Kartu profil memakai position: relative supaya badge status bisa diposisikan secara tepat di dalam kartu. Navbar dibuat sticky agar tetap terlihat saat halaman di-scroll, sedangkan tombol “Back to Top” memakai position: fixed supaya selalu muncul di pojok kanan bawah layar. Saya juga menambahkan z-index agar elemen yang penting berada di depan dan tidak tertutup elemen lain.

3.2. Tugas 2 Flexbox => Halaman Landing Sederhana

Buat landing page dengan struktur berikut menggunakan Flexbox: Navbar: Logo di kiri, menu di kanan, menggunakan justify-content: space-between Hero Section: Teks di kiri, gambar/ilustrasi di kanan menggunakan Flexbox Card Grid: Minimal 3 kartu produk/fitur yang menggunakan flex-wrap: wrap Footer: 3 kolom dengan Flexbox dan gap ★ Bonus: Tambahkan @media query agar layout berubah menjadi satu kolom di layar kecil.

a) Landing_page

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
  <title>Landing Page Flexbox</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>

  <!-- NAVBAR -->
  <nav class="navbar">
    <h2 class="logo">FlexBrand</h2>
    <ul class="nav-menu">
      <li>Home</li>
      <li>Features</li>
      <li>Contact</li>
    </ul>
  </nav>

  <!-- HERO SECTION -->
  <section class="hero">
    <div class="hero-text">
      <h1>Build Something Amazing</h1>
      <p>Flexbox makes layouting easier,
powerful, and responsive.</p>
      <button class="cta">Get Started</button>
    </div>

    <div class="hero-img">
      
    </div>
  </section>

  <!-- CARD GRID -->
  <section class="card-section">
```

```
        <div class="card">
            <h3>Fast</h3>
            <p>Our system lets you move quickly with
ease.</p>
        </div>
        <div class="card">
            <h3>Responsive</h3>
            <p>Automatically adapts to all screen
sizes.</p>
        </div>
        <div class="card">
            <h3>Modern</h3>
            <p>Built with simplicity and efficiency in
mind.</p>
        </div>
    </section>

    <!-- FOOTER -->
    <footer class="footer">
        <div class="footer-col">
            <h4>Company</h4>
            <p>About Us</p>
            <p>Careers</p>
        </div>

        <div class="footer-col">
            <h4>Services</h4>
            <p>Design</p>
            <p>Development</p>
        </div>

        <div class="footer-col">
            <h4>Support</h4>
            <p>Help Center</p>
            <p>Privacy Policy</p>
        </div>
    </footer>

</body>
</html>
```

b) style.css

```
.navbar {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 20px;
  background: #1f1f1f;
  color: white;
}

.nav-menu {
  display: flex;
  gap: 20px;
  list-style: none;
}

/* HERO SECTION */
.hero {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 60px;
  gap: 40px;
}

.hero-text {
  max-width: 50%;
}

.hero-text h1 {
  font-size: 2.5em;
}

.cta {
  padding: 10px 20px;
  font-size: 16px;
  margin-top: 15px;
  background: royalblue;
  color: white;
  border: none;
  cursor: pointer;
}
```

```
        border-radius: 6px;
    }

    .card-section {
        display: flex;
        gap: 20px;
        padding: 40px 60px;
        flex-wrap: wrap;
    }

    .card {
        flex: 1;
        min-width: 220px;
        padding: 20px;
        background: #f4f4f4;
        border-radius: 12px;
    }

    .footer {
        display: flex;
        gap: 40px;
        padding: 40px;
        background: #1f1f1f;
        color: white;
        justify-content: center;
    }

    .footer-col h4 {
        margin-bottom: 10px;
    }

    @media (max-width: 768px) {
        .hero {
            flex-direction: column;
            text-align: center;
        }

        .hero-text {
            max-width: 100%;
        }
    }
}
```

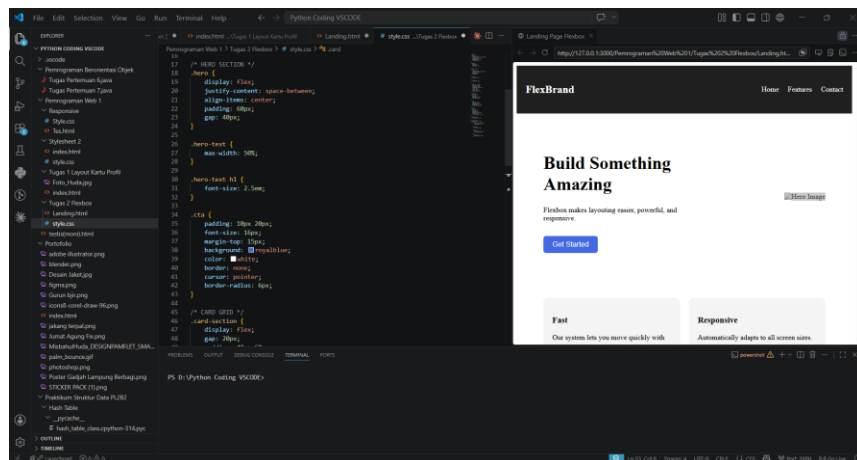
```

.footer {
  flex-direction: column;
  text-align: center;
}

.footer-col {
  margin-bottom: 15px;
}
}

```

Dengan output sebagai berikut:



Gambar 2.2 Tampilan output landing page

Flexbox digunakan pada landing page untuk mempermudah pengaturan tata letak elemen agar lebih rapi dan responsif. Pada bagian navbar, Flexbox membantu menempatkan logo di sisi kiri dan menu di sisi kanan menggunakan `justify-content: space-between`, sehingga kedua elemen tersusun dalam satu baris dengan

jarak yang seimbang. Di hero section, Flexbox digunakan untuk menempatkan teks dan gambar berdampingan secara horizontal dengan jarak yang proporsional, membuat tampilan lebih modern dan mudah dibaca. Pada card grid, Flexbox mengatur tiga kartu fitur agar tersusun rapi dalam satu baris, dan dengan flex-wrap: wrap, kartu-kartu tersebut dapat turun ke baris berikutnya ketika layar mengecil sehingga layout tetap responsif. Footer juga memanfaatkan Flexbox untuk membuat tiga kolom yang tersusun sejajar dan memiliki jarak yang konsisten. Media query kemudian digunakan untuk mengubah struktur menjadi satu kolom pada layar kecil agar tampilan tetap nyaman bagi pengguna mobile. Secara keseluruhan, Flexbox memberikan kemudahan dalam menyusun layout yang fleksibel dan adaptif tanpa memerlukan teknik layout yang rumit.

3.3. Tugas 3 Animation & Transition => Komponen Interaktif

Buat halaman berisi komponen interaktif berikut: Minimal 3 tombol dengan efek hover berbeda menggunakan transition (warna, transform, box-shadow) Sebuah loading spinner menggunakan @keyframes dan animation Sebuah elemen yang fade in saat halaman dibuka menggunakan animasi opacity Gunakan CSS Variables untuk mendefinisikan palet warna dan gunakan di seluruh file Bonus: Tambahkan animasi pada kartu yang berputar saat di-hover menggunakan transform: rotateY().

```
<!DOCTYPE html>
<html lang="id">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
  <title>Animasi & Transisi – Komponen Interaktif</title>
```

```
<style>
:root {
  --clr-bg: #f5f4f0;
  --clr-surface: #ffffff;
  --clr-text: #1c1b18;
  --clr-muted: #6b6a65;
  --clr-accent: #2e6fdc;
  --clr-accent-dark: #1a4fa8;
  --clr-success: #16803c;
  --clr-warn: #b45309;
  --clr-border: #e2e0da;
  --clr-shadow: rgba(0, 0, 0, 0.14);
}

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
}

body {
  font-family: 'Georgia', serif;
  background: var(--clr-bg);
  color: var(--clr-text);
  padding: 2.5rem 1.5rem;
  min-height: 100vh;
}

.page-title {
  font-size: 22px;
  font-weight: 700;
  letter-spacing: -0.02em;
  margin-bottom: 0.25rem;
}

.page-sub {
  font-size: 13px;
  color: var(--clr-muted);
  margin-bottom: 2rem;
}
```

```
.wrapper {
  max-width: 640px;
  margin: 0 auto;
}

.section {
  background: var(--clr-surface);
  border: 1px solid var(--clr-border);
  border-radius: 12px;
  padding: 1.5rem;
  margin-bottom: 1rem;
}

.section-label {
  font-size: 11px;
  font-weight: 400;
  letter-spacing: 0.12em;
  text-transform: uppercase;
  color: var(--clr-muted);
  margin-bottom: 1rem;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(14px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

.page-title,
.page-sub {
  animation: fadeIn 0.5s ease both;
}

.page-sub { animation-delay: 0.08s; }

.section {
```

```
    animation: fadeIn 0.55s ease both;
  }

.section:nth-child(1) { animation-delay: 0.15s; }
.section:nth-child(2) { animation-delay: 0.25s; }
.section:nth-child(3) { animation-delay: 0.35s; }
.section:nth-child(4) { animation-delay: 0.45s; }
.section:nth-child(5) { animation-delay: 0.55s; }

.btn-row {
  display: flex;
  gap: 10px;
  flex-wrap: wrap;
}

.btn {
  font-family: inherit;
  font-size: 13px;
  border-radius: 8px;
  padding: 9px 22px;
  cursor: pointer;
  border: 1.5px solid transparent;
  font-weight: 500;
  transition: all 0.25s ease;
  outline: none;
}

.btn-color {
  background: var(--clr-accent);
  color: #fff;
  border-color: var(--clr-accent);
}

.btn-color:hover {
  background: var(--clr-accent-dark);
  border-color: var(--clr-accent-dark);
}

.btn-scale {
  background: transparent;
  color: var(--clr-success);
  border-color: var(--clr-success);
}
```

```
}  
.btn-scale:hover {  
  transform: scale(1.09);  
  background: var(--clr-success);  
  color: #fff;  
}  
  
.btn-shadow {  
  background: var(--clr-surface);  
  color: var(--clr-warn);  
  border-color: var(--clr-warn);  
}  
.btn-shadow:hover {  
  box-shadow: 0 8px 20px var(--clr-shadow);  
  transform: translateY(-3px);  
}  
  
@keyframes spin {  
  to { transform: rotate(360deg); }  
}  
  
.spinner-wrap {  
  display: flex;  
  align-items: center;  
  gap: 14px;  
}  
  
.spinner {  
  width: 34px;  
  height: 34px;  
  border: 3px solid var(--clr-border);  
  border-top-color: var(--clr-accent);  
  border-radius: 50%;  
  animation: spin 0.85s linear infinite;  
  flex-shrink: 0;  
}  
  
.spinner-label {  
  font-size: 13px;  
  color: var(--clr-muted);
```

```
}

.fade-info {
  font-size: 13px;
  color: var(--clr-muted);
  line-height: 1.65;
}

.fade-info code {
  font-family: monospace;
  background: var(--clr-bg);
  padding: 1px 6px;
  border-radius: 4px;
  color: var(--clr-text);
  font-size: 12px;
}

.flip-scene {
  width: 190px;
  height: 100px;
  perspective: 600px;
  cursor: pointer;
}

.flip-card {
  width: 100%;
  height: 100%;
  position: relative;
  transform-style: preserve-3d;
  transition: transform 0.55s ease;
}

.flip-scene:hover .flip-card {
  transform: rotateY(180deg);
}

.flip-front,
.flip-back {
  position: absolute;
  inset: 0;
  border-radius: 10px;
```

```

        backface-visibility: hidden;
        display: flex;
        align-items: center;
        justify-content: center;
        font-size: 13px;
        font-weight: 500;
        border: 1.5px solid var(--clr-border);
        padding: 12px;
        text-align: center;
    }

    .flip-front {
        background: var(--clr-accent);
        color: #fff;
        border-color: var(--clr-accent);
    }

    .flip-back {
        background: var(--clr-surface);
        color: var(--clr-accent);
        transform: rotateY(180deg);
        border-color: var(--clr-accent);
    }

    .hint {
        font-size: 11px;
        color: var(--clr-muted);
        margin-top: 10px;
    }
</style>
</head>
<body>
    <div class="wrapper">

        <p class="page-title">Animasi &amp; Transisi</p>
        <p class="page-sub">Komponen interaktif dengan CSS
        Variables, @keyframes, dan transition.</p>

        <div class="section">
            <p class="section-label">Tombol – transisi hover</p>
            <div class="btn-row">

```

```

        <button class="btn btn-color">Warna</button>
        <button class="btn btn-scale">Scale</button>
        <button class="btn btn-shadow">Shadow Lift</button>
    </div>
</div>

<div class="section">
    <p class="section-label">Loading Spinner –
@keyframes</p>
    <div class="spinner-wrap">
        <div class="spinner" role="status" aria-
label="Memuat"></div>
        <span class="spinner-label">Memuat data...</span>
    </div>
</div>

<div class="section">
    <p class="section-label">Fade-in saat halaman
dibuka</p>
    <p class="fade-info">
        Semua elemen di halaman ini muncul dengan animasi
        <code>fadeIn</code> – opacity dari <code>0</code>
ke <code>1</code>
        disertai gerak naik ringan
(<code>translateY</code>), masing-masing
        diberi <code>animation-delay</code> berbeda agar
muncul bertahap.
    </p>
</div>

<div class="section">
    <p class="section-label">Flip Card rotateY()</p>
    <div class="flip-scene" role="img" aria-label="Kartu
yang bisa dibalik saat dihover">
        <div class="flip-card">
            <div class="flip-front">Hover saya</div>
            <div class="flip-back">Sisi belakang ✦</div>
        </div>
    </div>
    <p class="hint">Arahkan kursor ke kartu untuk melihat
efek flip.</p>

```



```

</div>

<div class="section">
  <p class="section-label">CSS Variables – palet
warna</p>
  <p class="fade-info" style="margin-bottom: 12px;">
    Semua warna didefinisikan di <code>:root</code> dan
dipakai di seluruh file.
  </p>
  <div style="display: flex; flex-wrap: wrap; gap:
8px;">
    <div style="display: flex; align-
items: center; gap: 6px; font-size: 12px; color: var(--clr-
muted);">
      <span style="width: 16px; height: 16px; border-
radius: 4px; background: var(--clr-accent); display: inline-
block;"></span>--clr-accent
    </div>
    <div style="display: flex; align-
items: center; gap: 6px; font-size: 12px; color: var(--clr-
muted);">
      <span style="width: 16px; height: 16px; border-
radius: 4px; background: var(--clr-success); display: inline-
block;"></span>--clr-success
    </div>
    <div style="display: flex; align-
items: center; gap: 6px; font-size: 12px; color: var(--clr-
muted);">
      <span style="width: 16px; height: 16px; border-
radius: 4px; background: var(--clr-warn); display: inline-
block;"></span>--clr-warn
    </div>
    <div style="display: flex; align-
items: center; gap: 6px; font-size: 12px; color: var(--clr-
muted);">
      <span style="width: 16px; height: 16px; border-
radius: 4px; background: var(--clr-bg); border: 1px solid var(--
clr-border); display: inline-block;"></span>--clr-bg
    </div>
    <div style="display: flex; align-
items: center; gap: 6px; font-size: 12px; color: var(--clr-

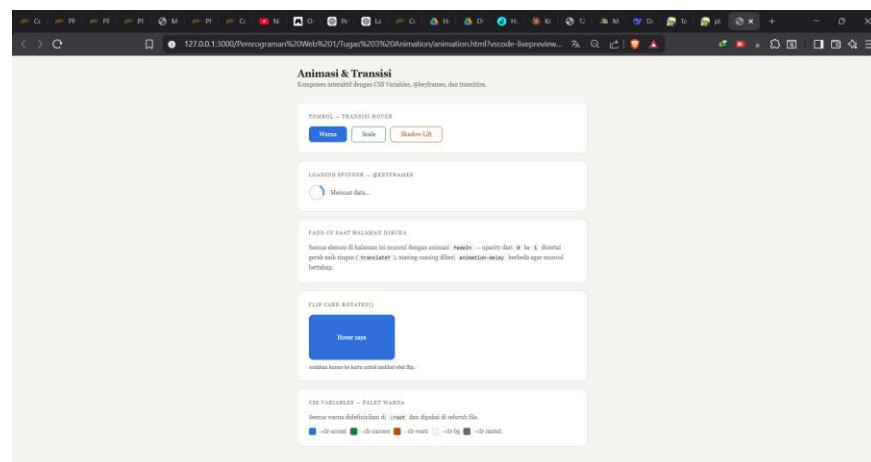
```

```

muted);">
    <span style="width:16px;height:16px;border-
radius:4px;background:var(--clr-muted);display:inline-
block;"></span>--clr-muted
  </div>
</div>
</div>
</div>
</div>
</body>
</html>

```

Dengan outputnya sebagai berikut:



Gambar 2.3 Tampilan output Animation

Halaman ini dibangun menggunakan HTML dan CSS murni tanpa library tambahan. Seluruh palet warna didefinisikan melalui CSS Variables di dalam :root variabel seperti --clr-accent, --clr-success, dan --clr-warn digunakan secara konsisten di seluruh file sehingga perubahan tema cukup dilakukan di satu tempat. Tiga tombol yang

tersedia masing-masing mendemonstrasikan efek transition yang berbeda: tombol pertama mengubah warna background, tombol kedua membesar dengan transform: scale(), dan tombol ketiga terangkat menggunakan kombinasi translateY dan box-shadow. Loading spinner dibuat dari elemen lingkaran yang dianimasikan dengan @keyframes spin berputar 360 derajat secara terus-menerus, sementara efek fade-in saat halaman dibuka menggunakan @keyframes fadeIn yang menganimasikan opacity dari nol ke satu sekaligus menggeser elemen ke atas masing-masing kartu diberi animation-delay berbeda agar muncul bertahap. Sebagai bonus, terdapat flip card yang memanfaatkan transform: rotateY(180deg) untuk menciptakan efek membalik kartu tiga dimensi. Agar bekerja dengan benar, elemen induk diberi perspective untuk kedalaman ruang, transform-style: preserve-3d agar elemen anak tetap berada dalam konteks 3D, dan backface-visibility: hidden pada sisi depan maupun belakang supaya sisi yang tidak menghadap layar benar-benar tersembunyi. Hasilnya adalah animasi flip yang tampak seperti membalik kartu fisik sungguhan saat kursor diarahkan ke atasnya.

4. Kesimpulan

Stylesheet, khususnya melalui penggunaan CSS, merupakan komponen penting dalam proses pembuatan website karena memisahkan struktur dan konten dari aspek visual. Dengan adanya stylesheet, pengembang dapat mengatur tampilan seperti warna, tipografi, layout, serta efek interaktif tanpa harus mencampurkannya dengan kode HTML. Pemisahan ini tidak hanya membuat kode lebih bersih dan mudah dipelihara, tetapi juga memungkinkan konsistensi desain di seluruh halaman website. Selain itu, stylesheets mendukung pengembangan antarmuka yang responsif, modern, dan user-friendly.

Dalam implementasinya, stylesheet memberikan fleksibilitas yang besar melalui berbagai fitur seperti selector, box model, Flexbox, Grid, hingga animation dan transition. Fitur-fitur tersebut memungkinkan pengembang merancang tampilan visual yang dinamis dan adaptif terhadap berbagai ukuran perangkat. Penggunaan stylesheet juga mempermudah proses pemeliharaan dan pengembangan lanjutan karena perubahan tampilan dapat dilakukan hanya dengan mengedit satu berkas CSS. Secara keseluruhan, stylesheet adalah fondasi utama dalam menciptakan pengalaman pengguna yang menarik, terstruktur, dan efisien pada sebuah website.

5. Daftar Pustaka

- [1] W3C, *Cascading Style Sheets Level 2 Revision 1 (CSS2.1) Specification*.
- [2] W3C, *Design Principles: Separation of Structure and Presentation*.
- [3] Håkon Wium Lie dan Bert Bos, *Cascading Style Sheets: Designing for the Web*.
- [4] W3C, *HTML 4.01 Specification – Style Sheets*.
- [5] E. Meyer, *CSS: The Definitive Guide*, O'Reilly Media.
- [6] W3C, *CSS Cascading and Inheritance Level 3*.
- [7] W3C, *Visual Formatting Model – CSS Box Model*.